© The Hong Kong Polytechnic University

THE HONG KONG POLYTECHNIC UNIVERSITY

DEPARTMENT OF COMPUTING

EXAMINATION

Subject : COMP3438 System Programming

Session : 2024 / 2025 Semester I

Date : 13 December 2024 Time : 12:30 - 14:30

Time Allowed: 2 Hours

Subject Examiner(s): Dr WANG Qixin, Dr SAXENA Divya

This question paper has a total of ___13___ pages (cover included).
(Some pages may be intentionally omitted.)

Instructions to Candidates:

Closed books and notes.

Answer ALL questions on the answer book.

Do not turn over the page until you are told to do so!

© The Hong Kong Polytechnic University

Part A

Question 1 [15%]

Read the following source code `led1driver.c` for implementing an LED driver, and `led1app.c` for the corresponding user level application. For simplicity, all exception handling codes are ignored, and you can assume no exception will happen in runtime.

The purpose of `led1app.c` is to repetitively ask for user input, to turn on/off an LED through the LED driver.

(a) [3%] Assume `led1driver.c` is bug-less. There is one and only one bug in `led1app.c` involving one and only one line of code. That buggy line is line _____ (fill in a value from 00 to 22) in `led1app.c`.

led1app.c

```
00 #include <sys/types.h>
01 #include <sys/stat.h>
02 #include <fcntl.h>
03 #include <unistd.h>
04 #include <stdio.h>
05 #include <stdlib.h>
06
07 void main() {
08     int fd;
09     fd = open("/dev/led1", O_WRONLY);
10     char cmd;
11     do {
12         printf("Input \'1\' to turn on the LED1, \n");
13         printf("      \'0\' to turn off, \n");
14         printf("      \'2\' to exit.\n");
15         scanf(" %c", cmd);
16         printf("Inputted \'%c\'.\n", cmd);
17         if (cmd == '2') break;
18         write(fd, &cmd, 1);
19     } while (1);
20     close(fd);
21     exit(EXIT_SUCCESS);
22 }
```

© The Hong Kong Polytechnic University

led1driver.c

```
00 #include <linux/module.h>
01 #include <linux/init.h>
02 #include <linux/fs.h>
03 #include <linux/types.h>
04 #include <linux/printk.h>
05 #include <linux/kdev_t.h>
06 #include <linux/cdev.h>
07 #include <linux/string.h>
08 #include <linux/gpio.h>
09 #include <asm-generic/uaccess.h>
10 #include <mach/gpio.h>
11
12 #define S_N 0
13 #define N_D 2
14 #define DRIVER_NAME "LED1 driver"
15
16 #define LED1 S5PV210_GPJ2(0)
17 #define LED_ON 0
18 #define LED_OFF 1
19
20
21 static dev_t apricot;
22
23 int orange(struct file *fp, const char __user *buf,
   size_t count, loff_t *position) {
24     char led_status;
25     int ret;
26     ret = copy_from_user(&led_status, buf, 1);
27     if (led_status == '1') {
28         gpio_set_value(LED1, LED_ON);
29     } else {
30         gpio_set_value(LED1, LED_OFF);
31     }
32     return 1;
33 }
34
```

(to be continued on the next page ...)

© The Hong Kong Polytechnic University

(continued from the previous page.)

```
35 int plum(struct inode* inodep, struct file *fp) {
36     _____;
37     return 0;
38 }
39
40 int apple(struct inode* inodep, struct file *fp) {
41     _____;
42     return 0;
43 }
44
45 struct file_operations grape = {
46     owner: THIS_MODULE,
47     open: apple,
48     write: orange,
49     release: plum,
50 };
51
52 struct cdev mellon;
53
54 static void _____ cherry(void) {
55     cdev_del(&mellon);
56     unregister_chrdev_region(apricot, N_D);
57 }
58
59 static int _____ pear(void) {
60     int ret;
61     ret = alloc_chrdev_region(&apricot, S_N, N_D,
   DRIVER_NAME);
62     cdev_init(&mellon, &grape);
63     mellon.owner = THIS_MODULE;
64     ret = cdev_add(&mellon, apricot, N_D);
65     int major = MAJOR(___);
66     return 0;
67 }
68
69 _____(pear);
70 _____(cherry);
71
72 MODULE_LICENSE("GPL");
73 MODULE_AUTHOR("Qixin Wang");
74 MODULE_DESCRIPTION("LED1 Char Driver");
```

© The Hong Kong Polytechnic University

(b) [8%]

Suppose all printed messages are truthful. Particularly, when the LED character device file is opened and closed, a corresponding message will be printed to indicate the open/close of the character device file.

Please fill in the blanks (the number of '_'s has nothing to do with the correct answer's string length) in `led1driver.c` respectively with one and only one of the following choices (these choices are listed alphabetically for your convenience).

A. `THIS_MODULE`
B. `__exit`
C. `__init`
D. `__user`
E. `apple`
F. `apricot`
G. `cherry`
H. `copy_from_user`
I. `copy_to_user`
J. `grape`
K. `mellon`
L. `module_exit`
M. `module_init`
N. `orange`
O. `pear`
P. `plum`
Q. `printk`
R. `printk("Device " DRIVER_NAME " closed.\n");`
S. `printk("Device " DRIVER_NAME " opened.\n");`
T. `read`
U. `write`

(b.1) [1%] For the blank of `led1driver.c` line `36`, we should fill in _____ (choose one from 'A', 'B', ..., 'U').

(b.2) [1%] For the blank of `led1driver.c` line `41`, we should fill in _____ (choose one from 'A', 'B', ..., 'U').

(b.3) [1%] For the blank of `led1driver.c` line `54`, we should fill in _____ (choose one from 'A', 'B', ..., 'U').

(b.4) [1%] For the blank of `led1driver.c` line `59`, we should fill in _____ (choose one from 'A', 'B', ..., 'U').

(b.5) [1%] For the blank of `led1driver.c` line `69`, we should fill in _____ (choose one from 'A', 'B', ..., 'U').

(b.6) [1%] For the blank of `led1driver.c` line `70`, we should fill in _____ (choose one from 'A', 'B', ..., 'U').

(b.7) [2%] For the blank of `led1driver.c` line `65`, we should fill in _____ (choose one from 'A', 'B', ..., 'U').

© The Hong Kong Polytechnic University

(c) [2%] There is one and only one line of `led1driver.c` that informs the Linux kernel on where to find the functions to handle open/close/write system calls upon the LED character device file. That line is line _____ (fill in a value from `00` to `74`) of `led1driver.c`.

(d) [2%] There is one and only one line of `led1driver.c` that finds the available major and minor number(s) for the the LED character device file. That line is line _____ (fill in a value from `00` to `74`) of `led1driver.c`.

References for Question 1

The `man` page for `close` is as follows.

```
CLOSE(2)                Linux Programmer's Manual                CLOSE(2)

NAME
       close - close a file descriptor

SYNOPSIS
       #include <unistd.h>

       int close(int fd);

DESCRIPTION
       close()  closes  a file descriptor, so that it no longer refers
       to any file and may be reused.  Any record locks (see fcntl(2))
       held  on  the  file  it  was  associated with, and owned by the
       process, are removed (regardless of the  file  descriptor  that
       was used to obtain the lock).

       If  fd  is the last file descriptor referring to the underlying
       open file description (see open(2)), the  resources  associated
       with  the open file description are freed; if the file descrip-
       tor was the last reference to a file which has been removed us-
       ing unlink(2), the file is deleted.
...
```

The `man` page for `exit` is as follows.

```
EXIT(3)                 Linux Programmer's Manual                EXIT(3)

NAME
       exit - cause normal process termination

SYNOPSIS
       #include <stdlib.h>

       void exit(int status);

DESCRIPTION
       The  exit()  function causes normal process termination and the
```

© The Hong Kong Polytechnic University

```
       value of status & 0xFF is returned to the parent (see wait(2)).

       All functions registered with atexit(3) and on_exit(3) are
       called, in the reverse order of their registration. (It is
       possible for one of these functions to use atexit(3) or
       on_exit(3) to register an additional function to be executed
       during exit processing; the new registration is added to the
       front of the list of functions that remain to be called.) If
       one of these functions does not return (e.g., it calls
       _exit(2), or kills itself with a signal), then none of the re-
       maining functions is called, and further exit processing (in
       particular, flushing of stdio(3) streams) is abandoned. If a
       function has been registered multiple times using atexit(3) or
       on_exit(3), then it is called as many times as it was regis-
       tered.

       All open stdio(3) streams are flushed and closed. Files cre-
       ated by tmpfile(3) are removed.

       The C standard specifies two constants, EXIT_SUCCESS and
       EXIT_FAILURE, that may be passed to exit() to indicate success-
       ful or unsuccessful termination, respectively.

RETURN VALUE
       The exit() function does not return.
...
```

The man page for `fprintf` is as follows.

```
PRINTF(3)                Linux Programmer's Manual                PRINTF(3)

NAME
       printf, fprintf, dprintf, sprintf, snprintf, vprintf, vfprintf,
       vdprintf, vsprintf, vsnprintf - formatted output conversion

SYNOPSIS
       #include <stdio.h>

       int printf(const char *format, ...);
       int fprintf(FILE *stream, const char *format, ...);
       int dprintf(int fd, const char *format, ...);
       int sprintf(char *str, const char *format, ...);
...
DESCRIPTION
       The functions in the printf() family produce output according
       to a format as described below. The functions printf() and
       vprintf() write output to stdout, the standard output stream;
       fprintf() and vfprintf() write output to the given output
       stream; sprintf(), snprintf(), vsprintf() and vsnprintf() write
       to the character string str.
...
```

© The Hong Kong Polytechnic University

The man page for fscanf is as follows.

```
SCANF(3)                  Linux Programmer's Manual                 SCANF(3)

NAME
       scanf, fscanf, sscanf, vscanf, vsscanf, vfscanf - input format
       conversion

SYNOPSIS
       #include <stdio.h>

       int scanf(const char *format, ...);
       int fscanf(FILE *stream, const char *format, ...);
       int sscanf(const char *str, const char *format, ...);

       #include <stdarg.h>

       int vscanf(const char *format, va_list ap);
       int vsscanf(const char *str, const char *format, va_list ap);
       int vfscanf(FILE *stream, const char *format, va_list ap);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       vscanf(), vsscanf(), vfscanf():
           _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L

DESCRIPTION
       The scanf() family of functions scans input according to format as
       described below. This format may contain conversion specifications; the results from such conversions, if any, are stored in
       the locations pointed to by the pointer arguments that follow format. Each pointer argument must be of a type that is appropriate
       for the value returned by the corresponding conversion specification.
...
```

The man page for open is as follows.

```
OPEN(2)                   Linux Programmer's Manual                 OPEN(2)

NAME
       open, openat, creat - open and possibly create a file

SYNOPSIS
       #include <sys/types.h>
       #include <sys/stat.h>
       #include <fcntl.h>
```

© The Hong Kong Polytechnic University

```
       int open(const char *pathname, int flags);
       int open(const char *pathname, int flags, mode_t mode);

...

DESCRIPTION
       The open() system call opens the file specified by pathname.
       If the specified file does not exist, it may optionally (if
       O_CREAT is specified in flags) be created by open().

       The return value of open() is a file descriptor, a small, non-
       negative integer that is used in subsequent system calls
       (read(2), write(2), lseek(2), fcntl(2), etc.) to refer to the
       open file.  The file descriptor returned by a successful call
       will be the lowest-numbered file descriptor not currently open
       for the process.
...
       The argument flags must include one of the following access
       modes: O_RDONLY, O_WRONLY, or O_RDWR.  These request opening
       the file read-only, write-only, or read/write, respectively.
...
```

The man page for `printf` is as follows.

```
See fprintf.
```

The man page for `scanf` is as follows.

```
See fscanf.
```

The man page for `write` is as follows.

```
WRITE(2)                  Linux Programmer's Manual                 WRITE(2)

NAME
       write - write to a file descriptor

SYNOPSIS
       #include <unistd.h>

       ssize_t write(int fd, const void *buf, size_t count);

DESCRIPTION
       write()  writes  up to count bytes from the buffer starting at buf
       to the file referred to by the file descriptor fd.

       The number of bytes written may be less than count if,  for  exam-
       ple,  there  is  insufficient  space  on  the  underlying physical
```

© The Hong Kong Polytechnic University

medium, or the RLIMIT_FSIZE resource limit is encountered (see setrlimit(2)), or the call was interrupted by a signal handler after having written less than count bytes. (See also pipe(7).)

For a seekable file (i.e., one to which lseek(2) may be applied, for example, a regular file) writing takes place at the file offset, and the file offset is incremented by the number of bytes actually written. If the file was open(2)ed with O_APPEND, the file offset is first set to the end of the file before writing. The adjustment of the file offset and the write operation are performed as an atomic step.

POSIX requires that a read(2) that can be proved to occur after a write() has returned will return the new data. Note that not all filesystems are POSIX conforming.

According to POSIX.1, if count is greater than SSIZE_MAX, the result is implementation-defined; see NOTES for the upper limit on Linux.

RETURN VALUE

On success, the number of bytes written is returned. On error, -1 is returned, and errno is set to indicate the cause of the error.

Note that a successful write() may transfer fewer than count bytes. Such partial writes can occur for various reasons; for example, because there was insufficient space on the disk device to write all of the requested bytes, or because a blocked write() to a socket, pipe, or similar was interrupted by a signal handler after it had transferred some, but before it had transferred all of the requested bytes. In the event of a partial write, the caller can make another write() call to transfer the remaining bytes. The subsequent call will either transfer further bytes or may result in an error (e.g., if the disk is now full).

If count is zero and fd refers to a regular file, then write() may return a failure status if one of the errors below is detected. If no errors are detected, or error detection is not performed, 0 will be returned without causing any other effect. If count is zero and fd refers to a file other than a regular file, the results are not specified.

...

© The Hong Kong Polytechnic University

Part B

Question 2 [1%]
Match all items in Group 1 with correct options from those given in Group 2.
Group 1
P. Register allocation
Q. Pushdown automata
R. Regular expression
S. Dataflow analysis
Group 2
1. Syntax analysis
2. Code optimization
3. Lexical analysis
4. Code generation

Question 3 [1%]

Question 4 [1%]

Question 5 [2%]
In some programming languages, an identifier is permitted to be a letter followed by any combination of letters or digits. L and D denote the sets of letters and digits, respectively. Write a Regular Expression for this identifier definition.

Part C

Question 6 [2%+2%+1%]
Analyze the following C code segment and identify the correct statement(s):

```
for (i = 0; i < n; i++) {
  for (j = 0; j < n; j++) {
    if (i % 2) {
      x += (4 * j + 5 * i);
      y += (7 + 4 * j);
    }
  }
}
```

(a) Identify all loop-invariant computations in the code. Explain how these computations can be optimized.

© The Hong Kong Polytechnic University

(b) Discuss any scope for common sub-expression elimination in the given code segment.

(c) Analyze whether there is any potential for strength reduction in this code. If yes, then optimize the computationally expensive operations.

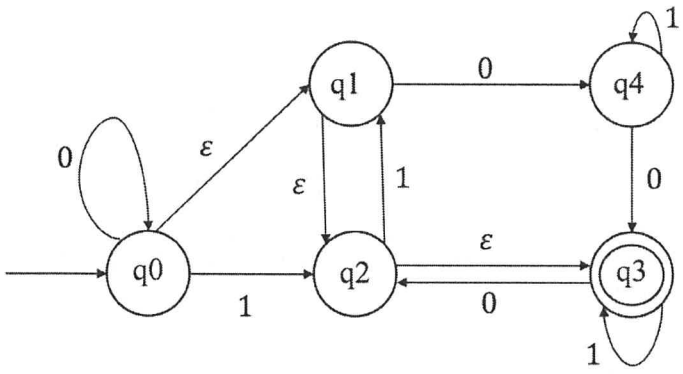
Question 7 [5%]
Consider the grammar E → E + n | E * n | n. For a sentence n + n * n, what are the handles in the right-sentential form of the reduction?

Question 8 [5%]
Consider the following grammar.
S → aSB| d
B → b
The number of reduction steps taken by a bottom-up parser while accepting the string 'aaadbbb' is _____.

Question 9 [5%]
Design a NFA with = {a, b} accepts the strings with an even number of *a*'s followed by *b*.

Question 10 [5%]
Design a NFA for the following regular expression p (p* qp* qp*)*.

Question 11 [5%+5%]
Convert the given ε-NFA into an NFA without null transitions.
(a) Compute the ε-closure for each state.
(b) Construct the NFA without null transitions.



Question 12 [5%]
Consider the following code segment.
x = u * t;
y = x * a;
x = y + w;
y = t - z;
y = x - y;
The minimum number of *total* variables required to convert the above code segment to *static single assignment* form is _____. Static Single Assignment is a form of intermediate representation in a compiler where every variable is assigned exactly once.

© The Hong Kong Polytechnic University

Part D

Question 13 [10%]
Given the grammar
E→E&E
E→E#E
E→E@E
E→E~E
E→−E
E→(E)
E→id

Operator Precedence Rules:
1. @ and ~ have higher precedence than & and #.
2. & and # have equal precedence and are left-associative.
3. @ and ~ have equal precedence and are left-associative.
4. Unary - (negation) has the highest precedence and is right-associative.
5. Parentheses ( ) have the highest precedence for grouping expressions.
6. Assume that ambiguity resolution has been handled and is not relevant for this task.

Non-terminal:
- E

Terminals:
- id
- Operators: &, #, @, ~, -
- Parentheses: (,)

Parse the input string using shift-reduce parsing, considering the operator precedence rules:

id @ id & id ~ id # id

Show all shift-reduce steps based on operator precedence and associativity. Show the stack, input, and action at each step.

Question 14 [5%+5%+5%+5%+10%]
S → aA | bA | cB | dB
A → Aa | Ad | d
B → Ba | Bb | C
C → cC | ε
where ε is the null symbol.
Perform the following tasks:
(a) Identify and eliminate any left recursion in the given grammar.
(b) Perform left factoring on the resulting grammar from part (a) if necessary. Provide an explanation for whether or not left factoring is required.
(c) Compute the First sets for all non-terminals in the grammar obtained after part (b).
(d) Compute the Follow sets for all non-terminals in the grammar obtained after part (b).
(e) Construct the LL(1) parsing table and determine if the final grammar is LL(1) or not.

END OF QUESTION PAPER